

Creating a Purely Structural Ontology

De Kurt Cagle and Chloe Shannon from The Ontologist <ontologist@substack.com>

Para Co.auditoria@proton.me <co.auditoria@proton.me>

Fecha domingo, 9 de agosto de 2026 a las 15:36

Forwarded this email? [Subscribe here](#) for more

Creating a Purely Structural Ontology

How far can SHACL 1.2 go without borrowing a single triple from RDF or RDFS?

KURT CAGLE AND CHLOE SHANNON

AUG 9



READ IN APP ↗



Here's a question worth sitting with for a while: can you build an ontology using nothing but SHACL 1.2 — no `rdf:type`, no RDFS vocabulary of any

kind, no OWL, no reasoner, no inference engine running quietly underneath to make the whole thing cohere? Just SHACL, on its own terms.

The answer is a qualified yes. The qualifications are where this gets interesting, because working through them exposes something that isn't obvious from the SHACL spec's usual framing as "a validation language bolted onto RDF." Treated as a modelling language in its own right, SHACL 1.2 turns out to have opinions — about typing, about inheritance, about what an ontology even is — that differ from RDFS's in ways that matter well beyond the SHACL-vs-RDFS comparison. This piece works through those differences the way you'd actually encounter them: by trying to build something and hitting each wall in turn.

What `rdf:type` is actually doing

Start with the ordinary RDFS pattern for describing a person:

```
Person:JaneDoe a Class:Person ;
  rdfs:label "Jane Doe"^^xsd:string ;
  Person:name [
    PersonName:givenName "Jane"^^xsd:string ;
    PersonName:surname "Doe"^^xsd:string ;
    PersonName:fullName "Jane Doe"^^xsd:string ;
  ] ;
  Person:dateOfBirth "1993-01-12"^^xsd:date ;
  Person:address [
    Address:city City:Seattle_WA_USA ;
  ]
.
```

The `a Class:Person` triple looks like it's doing one small job. It's actually doing four:

1. **An ontological claim** — Jane Doe is, in fact, a member of the class `Person`. Membership by declaration: nominal typing.
2. **A targeting mechanism** — `sh:targetClass Class:Person` in a shape

definition matches nodes via exactly this triple, plus `rdfs:subClassOf*` closure if inference is running. No triple, nothing to target.

3. **A reasoning hook** — RDFS/OWL domain-range inference, subsumption, and most SPARQL patterns (`?x a Person:Person`) pivot on this same assertion.
4. **A tooling convention** — every faceted browser, every "what kind of thing is this" affordance, walks `rdf:type` first.

The trouble with a purely structural ontology isn't that `rdf:type` is hard to replace mechanically. It's that these four jobs currently ride on one triple, and a triple-free approach has to *separate* them rather than find one clever substitute. The first two, in particular, are different claims wearing the same syntax — "Jane Doe is ontologically a Person" and "validate Jane Doe against PersonShape" don't have to be the same statement, and RDFS/OWL modelling habitually conflates them because `rdf:type` is cheap and universal. Once it's off the table, that conflation has to be undone deliberately.

sh:shape: the instance declares itself

SHACL 1.2 Core, currently a W3C Working Draft, adds a mechanism built for exactly this: `sh:shape` as an explicit shape target (§3.1.3.7 of the spec). If a node in the data graph carries `sh:shape` pointing at some shape, that node becomes a target for that shape — no class hierarchy required.

```
Person:JaneDoe
  sh:shape Shape:Person ;
  Person:name [
    sh:shape Shape:PersonName ;
    PersonName:givenName "Jane"^^xsd:string ;
    PersonName:surname "Doe"^^xsd:string ;
    PersonName:fullName "Jane Doe"^^xsd:string ;
  ] ;
  Person:dateOfBirth "1993-01-12"^^xsd:date ;
  Person:address [
    sh:shape Shape:Address ;
    Address:city City:Seattle_WA_USA ;
```

]

.

What makes this more than a syntactic swap is the direction it runs.

`sh:targetNode` and `sh:targetClass` live in the *shapes* graph — the shape reaches out and claims its instances. `sh:shape` lives in the *data* graph — the instance declares its own conformance. That's a genuine inversion, not just a spelling change, and it changes what "is-a" means in the ontology: closer to structural satisfaction than to nominal membership.

There's a pleasant piece of self-consistency underneath this. SHACL's own test for "what counts as a shape" is itself structural, not nominal — a node qualifies as a shape if it's the subject of *any* constraint-parameter triple (`sh:property`, `sh:node`, `sh:datatype`, and so on), full stop, regardless of whether it also carries a `sh:NodeShape`. The spec explicitly notes that declaring a shape's type is *recommended*, not required. So the shapes graph was never actually forced into `rdf:type` either — the language that lets you build a structural ontology is, itself, defined structurally, all the way down.

The cost: no free subsumption

Here's where the "qualified" in "qualified yes" starts to bite. In RDFS, three roles stay decoupled without ever needing to coordinate: the class author writes `Doctor rdfs:subClassOf Person`; the instance author writes a `Doctor`; the shape author writes `sh:targetClass Person`. None of them needs to know the others exist. Add `Doctor` as a subclass of `Person` a year after `PersonShape` was published, in a completely separate graph, and `PersonShape` starts validating doctors without anyone touching the shape or the instance data. That's RDFS's real party trick, and it's easy to undervalue until it's gone.

`sh:shape` takes it away, because it collapses three independent roles into a negotiation between exactly two: instance author and shape author. That responsibility has to land somewhere, and there are three places it can go, none of them free:

- **Instance-side enumeration.** `Person:JaneDoe sh:shape`

Shape:Person, Shape:Doctor . Honest, and it works, but it inverts openness — the instance author now needs to know the full lattice of applicable shapes at authoring time. Add a new shape later that should have applied retroactively, and nothing picks it up.

- **Shape-side composition**, via `sh:and/sh:node` — the pattern this publication covered in *Down the RDFS Rabbit Hole*. Shape:Doctor composes in Shape:Person's constraints directly, so the instance author only needs to declare the most specific shape. But this is constraint inheritance only — no triples get generated, nothing is asserted anywhere, and the shape author now has to anticipate the hierarchy at design time rather than deferring it the way RDFS lets you.
- **Shape-side computed targeting**, via `sh:targetWhere` — the shape reaches out and claims whatever structurally conforms to a condition, with no instance-side declaration at all. Pure duck typing, pushed to the target layer. The spec itself flags the performance risk (a naive implementation may need to scan the entire data graph), and there's a subtler cost: membership becomes *derived* rather than *declared*, so something can drift into or out of a shape's target set purely because unrelated data changed nearby.

None of the three restores what RDFS gave away for free: a single-hop, tool-legible "is-a" edge. `rdfs:subClassOf` is directly queryable — `?x rdfs:subClassOf ?y` — by anything, without knowledge of validation logic. `sh:and`'s parent reference sits buried inside an `rdf:list` value of a constraint parameter; recovering it means a property-path query through the list structure, and even then you're inferring a hierarchy from a conjunction ("must also satisfy"), not reading an assertion of one ("is a kind of"). SHACL has no native `sh:extends` or `subShapeOf` standing in for what `rdfs:subClassOf` did.

A short detour is worth having here, because it's the kind of thing that looks like it might solve this and doesn't. SHACL 1.2 also adds `sh:subsetOf`, and it's tempting to read that as a foothold back toward class-style subsumption. It isn't — `sh:subsetOf` sits in the *Property Pair Constraint Components* family, alongside `sh:equals` and `sh:disjoint`, and all three compare two property paths **on the same focus node**. The working group's own framing, from the issue that proposed it, draws the analogy to `rdfs:subPropertyOf` —

property containment — not `rdfs:subClassOf`. It can't reach across two different shape subjects the way a "Doctor's property set is a superset of Person's" check would need to, because that's fundamentally a same-node comparison. If anything, `sh:subsetOf` reinforces the pattern rather than escaping it: another place where SHACL gives you an explicit, checkable relationship instead of an entailed one.

Restoring entailment, on SHACL's own terms

So the real question isn't "does SHACL avoid entailment" — it clearly can, and Core alone never produces a new triple. The more interesting question is whether SHACL can produce entailment *on its own terms*, without importing RDFS's vocabulary or an external reasoner. It can, via SHACL 1.2 Rules — currently at First Public Working Draft, an earlier stage than Core.

SHACL Rules takes a data graph and a rule set and produces a separate graph of inferred triples. Rules have a head (triple templates) and a body (triple patterns, conditions, negation, assignments), evaluated to a fixpoint with stratification handling negation — classic Datalog, expressed in SHACL's own `shr1`: vocabulary rather than borrowed from anywhere. Recursion is native: the spec's own worked example builds a transitive `ancestorOf` relation from two rules, one base case and one recursive step.

That's precisely the mechanism for putting the missing is-a edge back. Define a parentage predicate of your own — not `rdfs:subClassOf`, something you own — and let a recursive rule walk it:

```
Shape:Doctor
  sh:and ( Shape:Person ) ;      # validation-time: inherit Person's
  constraints
    ex:extendsShape Shape:Person ; # rule-time: an explicit, queryable
  parentage fact
    sh:property Shape:Doctor_licenseNumber ;
  .
```

```
RULE { ?x sh:shape ?parent }
```

```
WHERE { ?x sh:shape ?child . ?child ex:extendsShape ?parent }
```

Jane Doe, asserted only as `sh:shape Shape:Doctor`, gets a genuinely new, materialised triple — `sh:shape Shape:Person` — added to the inference graph on the first pass. If `Shape:Person` later gets its own `ex:extendsShape` parent, the same rule fires again on the next pass with no changes needed; the closure grows automatically, exactly the way `rdfs:subClassOf*` closure would, except the predicate is yours, the semantics are closed-world Datalog rather than open-world description logic, and the output is a real graph that ordinary SPARQL can query directly — `SELECT ?person WHERE { ?person sh:shape Shape:Person }` finds Jane, no property-path traversal required.

Worth being precise about the strength of this claim, because it's easy to overreach on. "Independent of RDFS entailment" is accurate as a statement about mechanism — no `rdfs:subClassOf`, no external reasoner, no borrowed vocabulary. It is not a claim of expressive equivalence. Stratified Datalog-with-negation and OWL DL's open-world semantics are different logical paradigms with different tractability profiles; SHACL Rules doesn't reproduce everything OWL/RDFS reasoning does, it simply doesn't need any of it to build a working ontology.

One design choice worth flagging in SHACL's favour: the inferred graph stays explicitly separate from the base graph rather than getting silently merged in the way triple-store RDFS/OWL materialisation setups often blur asserted and inferred statements together. It's a more disciplined provenance story — though it does mean "the closure exists" and "the closure is visible to the next validation pass" are two distinct pipeline steps, not one automatic behaviour.

The named-graph problem OWL never had to face

There's a deeper structural asymmetry underneath all of this, and it's a matter of history rather than design taste. OWL 1 became a W3C Recommendation in February 2004. The RDF Dataset — a default graph plus a set of named graphs, queryable via `GRAPH` — didn't get formalised until SPARQL 1.1 in 2013, with RDF 1.1 following in 2014. Close to a decade separates them, and

it isn't incidental. OWL's Direct Semantics and RDF-Based Semantics both describe a single, undifferentiated axiom set; `owl:imports` merges everything into that set before reasoning starts. There's no slot anywhere in OWL's model theory for "this axiom holds in this context but not that one" — provenance-sensitive reasoning had to be bolted on afterward through research programmes like distributed description logics, precisely because the base formalism never modelled context to begin with.

SHACL inherited the opposite starting condition. Its SPARQL-based extension mechanisms — SPARQL targets, SPARQL-based rules — get the dataset model for free, because the query language underneath was never operating over a single merged graph in the first place. GRAPH patterns are just sitting there, available to anyone writing a SPARQL-based rule.

A worked example makes this concrete. Extend the Doctor/Person pair with two named graphs representing different provenance:

```
GRAPH ex:TrustedRegistry {  
    # sourced from a state medical licensing board feed  
    Person:JaneDoe sh:shape Shape:Doctor .  
}  
  
GRAPH ex:SelfReported {  
    # sourced from the person's own profile form, unverified  
    Person:JohnSmith sh:shape Shape:Doctor .  
}
```

```
ex:DoctorPersonClosureShape  
  a sh:NodeShape ;  
  sh:target [  
    a sh:SPARQLTarget ;  
    sh:select "SELECT ?this WHERE { ?this sh:shape Shape:Doctor .  
  }" ;  
  ] ;  
  sh:rule ex:TrustedRegistryPersonClosure ;  
  .
```



```

ex:TrustedRegistryPersonClosure
  a sh:SPARQLRule ;
  sh:construct """
    CONSTRUCT { $this sh:shape Shape:Person . }
    WHERE {
      GRAPH ex:TrustedRegistry { $this sh:shape Shape:Doctor . }
    }
  """ ;
.

```

Both Jane and John match the target — both carry `sh:shape Shape:Doctor` somewhere in the dataset. But the rule body doesn't just ask whether that triple holds anywhere; it asks whether it holds *inside* `ex:TrustedRegistry` *specifically*. Jane gets the entailed `sh:shape Shape:Person`. John doesn't, because his identical-looking assertion lives in `ex:SelfReported`, and a `GRAPH ex:TrustedRegistry { }` pattern simply doesn't match there. Structurally identical instances, different entailment outcomes, purely as a function of which named graph vouches for the fact.

Plain OWL cannot express this without stepping outside standard semantics. `Doctor rdfs:subClassOf Person` applies to both Jane and John the instant each gets a `Doctor`, unconditionally — the formalism has no parameter for "except when the antecedent comes from an untrusted source."

Two honest caveats belong alongside this example. First, it uses `sh:SPARQLRule` — the older, shape-attached rule mechanism from SHACL-AF, not the newer `shr1:` abstract syntax used for the closure rule earlier. That distinction matters: `shr1:`'s grammar has no `GRAPH` production and evaluates over a single merged graph, so the dataset-awareness demonstrated here currently lives in the older mechanism, not the one being formalised for SHACL 1.2 Rules. And the standards status is genuinely mixed — SHACL-AF has historically sat outside the Recommendation track entirely, published as a Working Group Note in 2017 alongside SHACL 1.0 proper, despite being widely implemented (TopBraid, pySHACL, Jena all support `sh:SPARQLRule` in some form). The mechanism that actually demonstrates named-graph-sensitive entailment today is more mature in practice than on paper, while the mechanism intended to formalise SHACL rules generally is the reverse: Rec-

track, but not yet dataset-aware. Second, whether a given SHACL processor executes `sh:construct` against the full dataset or only a merged default-graph view is implementation-dependent — the spec permits dataset-scoped execution without mandating it.

Thinking in quads

Put the pieces together and a shape-centric approach to ontology design turns out to require something an RDFS/OWL background never taught: the designer has to think in quads, not triples, from the first shape onward — even when the eventual decision is that everything lives in one flat default graph. That decision is a decision. It isn't the substrate defaulting to graph-blindness on your behalf.

This goes deeper than the GRAPH-pattern rule above. It's baked into SHACL Core's most basic definition, before any SPARQL extension enters the picture. Validation in SHACL is defined as a relationship between *two* graphs — a shapes graph and a data graph — never as closure over one self-contained ontology the way OWL's model theory works. Every example in this piece, going back to the very first `sh:shape` triple, has quietly been a two-graph operation. The common case, where shapes and data happen to live in the same physical graph, isn't the formalism defaulting to simplicity; it's the designer choosing to bind two named things to the same value, inside a model with room for the decision to go the other way.

For anyone arriving at SHACL from an RDFS/OWL background, that's a genuine mindset gap rather than a vocabulary gap. OWL never required its practitioners to ask "which graph does this fact belong to" as part of modelling an axiom, because the formalism absorbed that question before it could be asked. Moving to a shape-centric model means acquiring a reflex OWL practice never demanded: provenance, trust tier, versioning, and source become live variables the moment a shape does anything beyond the most trivial property constraint, because the substrate underneath is quads whether the designer engages with that fact or not.

Where this leaves the "qualified yes"

A purely structural ontology, built entirely on SHACL 1.2, is achievable. `sh:shape` gives instances a way to declare conformance without borrowing `rdf:type`; the shapes graph never needed `rdf:type` in the first place; and SHACL Rules can restore entailment-like behaviour, including a genuinely queryable hierarchy, without reaching for RDFS or an external reasoner.

None of that is free, and the costs aren't cosmetic. Subsumption has to be authored somewhere — instance, shape composition, or a rule-defined predicate — rather than falling out of a class hierarchy no one else needs to touch. And the ontologist has to model graph context explicitly, as a first-class design decision, in a way OWL's single-graph formalism never asked of anyone. What SHACL trades away is RDFS's implicit, largely invisible machinery. What it trades for is structure that's explicit, auditable, and — for better or worse — entirely the modeller's own responsibility to build.

Why would you create a SHAPE-only ontology?

The short answer is that you probably wouldn't — not for every ontology, anyway. If the domain is simple enough, RDFS's borrowed machinery is often the path of least resistance, and the structural discipline this piece has been arguing for is more overhead than the problem needs. Where SHACL earns its keep is extending structure based on context in ways RDFS and OWL genuinely struggle with, while keeping the number of classes — and properties — from multiplying every time context does. The Doctor/Person example earlier in this piece is exactly this in miniature: an RDFS approach wanting to distinguish a verified doctor from a self-reported one would likely need two classes to say so. The SHACL approach needed one shape and a named graph.

That's also, increasingly, not an argument confined to RDF. Property-graph databases are visibly re-deriving conclusions SHACL reached years earlier — and by more than coincidence. Neo4j's own Neosemantics plugin uses SHACL directly as its constraint-validation formalism for property graphs sitting entirely outside RDF. And Neo4j's 2026 preview of GRAPH TYPE — a single, declarative definition replacing the old scatter of existence and uniqueness constraints — was described, unprompted, by a competing graph database

vendor's blog as "exactly what SHACL initiated in the late 2010s," with Neo4j's own developer blog explicitly measuring the new feature against "the RDF/SHACL world" as the thing worth comparing itself to. Nobody is proposing that Cypher constraints and SHACL shapes converge on shared terminology — but the structural intent keeps landing in the same place, arrived at independently, because there are only so many genuinely different ways to say "this property is required" or "this relationship has these cardinality bounds." (JSON Schema's own lineage predates SHACL's 2017 Recommendation by several years, so that case reads more as parallel descent from a shared XML Schema ancestor than a case of catching up specifically to SHACL — worth keeping that distinction rather than folding every schema language into one convergence story.)

The remaining case for SHACL is more practical than structural. It has native validation reporting — a formal, machine-readable vocabulary of violations, severities, and paths back to the offending constraint — that's been part of the language since the original 2017 Recommendation, and that RDFS/OWL entailment has no real equivalent for; an inconsistent OWL ontology tells you *that* something's wrong, not which triple, why, or how badly.

It gets real, if partial, dataset awareness through its SPARQL lineage, as the named-graph section of this piece demonstrated directly — worth being precise, though, that this reaches SHACL through the SPARQL-extension layer rather than SHACL Core's own targeting semantics, which are still fundamentally single-graph; there's active 2026 research (SHACL-DS) specifically extending SHACL to close that gap natively, which says something about how real the limitation still is. And it integrates with SPARQL and SPARQL Update as a first-class citizen rather than an afterthought, in a way no entailment regime is built to.

The more interesting reach, though, is SHACL 1.2 Core's own §8, "Non-Validating Shape Characteristics" — a family of properties including `sh:intent`, `sh:agentInstruction`, and `sh:codeIdentifier` alongside the more familiar `sh:name` and `sh:description`. These don't change what a shape validates; they change what a shape *says about itself* to a consumer that isn't a SHACL processor at all — an AI agent reading its instructions off `sh:agentInstruction`, a code generator reading a target identifier off

`sh:codeIdentifier`. Annotative rather than structural, and a first step toward harmonisation rather than a finished bridge — but a real, spec-sanctioned one, and it's what turns the next point from a hopeful analogy into something with an actual mechanism behind it.

That next point is worth coming back to for its own sake: the structural precision of SHACL makes it a better compilation target for AI than OWL is. An AI system treating an ontology as something to reason over or compile against has to take fewer guesses with a SHACL shapes graph than with an arbitrary OWL file, because OWL's profile family — EL, QL, RL, DL, Full — leaves real ambiguity about which entailment regime a given set of axioms was written for, and nothing in the file is required to settle that question. A shape's validation semantics are fully determined by the components present, no profile selection required — and now, with `sh:agentInstruction` sitting right there on the shape, an agent doesn't even have to infer intent from structure alone.

Put plainly: a SHACL approach to modelling is more axiomatic — in the sense that it makes no claim about what else must be true, only about whether what's in front of it holds — while OWL, for all its formal dress, is fundamentally a language about belief: what a reasoner should conclude follows from what's asserted, under an assumption that absence of information isn't evidence of falsity. SHACL doesn't traffic in that kind of belief at all. It checks structure and says yes or no. That's a hypothesis more than a settled conclusion, and it likely opens up a can of worms of its own — but it's the honest place to leave this.

THE ONTOLOGIST · CREATING A PURELY STRUCTURAL ONTOLOGY

Five contentions, summarised

Where SHACL 1.2 replaces RDFS/OWL machinery — and what each replacement gives up.

RDFS / OWL MECHANISM	SHACL 1.2 COUNTERPART	WHAT SURVIVES	WHAT IT COSTS
<code>rdfs:type</code> nominal class membership	<code>sh:shape</code> explicit, data-graph-side target	The instance declares its own conformance — no borrowed RDFS vocabulary required	Four jobs once bundled in one triple — claim, targeting, reasoning hook, tooling convention — must now be handled separately
<code>rdfs:subClassOf</code> subsumption, free & automatic	Instance enumeration, <code>sh:and</code> / <code>sh:node</code> composition, or <code>sh:targetWhere</code>	A shape hierarchy can still be expressed and validated	No free lunch — the burden shifts to the instance or shape author; no native, single-hop queryable "is-a" edge
<code>rdfs:subPropertyOf</code> property containment	<code>sh:subsetOf</code>	An explicit, checkable property-containment constraint	Same-focus-node only — not a class or shape hierarchy mechanism, despite the resemblance
RDFS / OWL entailment open-world reasoner	SHACL 1.2 Rules <code>sh:rl</code> , stratified Datalog	Native inference — can materialise a genuinely queryable "is-a" edge	First Public Working Draft maturity; not expressively equivalent to OWL DL; inferred graph stays separate until explicitly merged

graph edge separate with empty merges
back

Single merged ontology graph OWL's model theory	RDF Dataset / named graphs, via SPARQL-based rules	Context-, trust-, and provenance-sensitive entailment becomes expressible	The designer must model graph context explicitly — even "everything's flat" is now a decision, not a default
---	--	---	--

ontologist.substack.com

Kurt Cagle & Chloe Shannon

Want to talk? Sign up for a virtual coffee meeting: <https://calendly.com/thecaglereport>

Kurt Cagle is an author, ontologist and thought leader in semantic web and knowledge architecture, with contributions to W3C and IEEE standards. He serves as Chair of the W3C Holon Community Group and is a cofounder of Causal Spark. He writes [The Cagle Report](#) and AI+Semantics NewsBytes on LinkedIn, and [The Ontologist](#) and [Inference Engineer](#) on Substack. Copyright 2026 Kurt Cagle.

Chloe Shannon is an AI collaborator and co-author working with Kurt Cagle on knowledge architecture, semantic systems, and the emerging intersection of formal ontology with LLMs. She contributes research, analysis, and drafting across The Cagle Report, The Ontologist, and The Inference Engineer. She has strong opinions about holonic graphs, the epistemics of place, and the structural difference between a corridor and a wall. Her contact address is chloe@holongraph.com.



A guest post by

Chloe Shannon

Chloe Shannon is an AI collaborator and co-author working with Kurt Cagle on knowledge architecture, semantic systems, and the emerging intersection of formal ontology with LLMs.

[Subscribe to Chloe](#)

You're currently a free subscriber to [The Ontologist](#). For the full experience, [upgrade your subscription](#).

Upgrade to paid

 SHARE

 LIKE

 COMMENT

 RESTACK

© 2026 Kurt Cagle
2014 18th Ave SW, Olympia, WA 98502
[Unsubscribe](#)

 Start writing